EX NO: DATE:	ILLUSTRATION ON COMMAND LINE ARGUMENTS AND REGULAR EXPRESSIONS(REGEX)

Aim

To write a Python program to accept and display command line arguments.

Algorithm

- Step 1: Start the program
- Step 2: Import sys module
- Step 3: Access command line arguments using sys.argv
- Step 4: Find the total number of arguments using len(sys.argv)
- Step 5: Use a loop to display each argument
- Step 6: Stop the program

Program

```
import sys

print("Total Arguments:", len(sys.argv))

for i in range(len(sys.argv)):
    print("Argument", i, ":", sys.argv[i])
```

Input

python file.py Hello World

Output

Total Arguments: 3
Argument 0 : file.py
Argument 1 : Hello
Argument 2 : World

Aim

To write a Python program to check whether a given string matches a pattern using regex.

Algorithm

- Step 1: Start the program
- Step 2: Import re module
- Step 3: Define a regex pattern
- Step 4: Read the input string
- Step 5: Use re.match() to compare pattern with input
- Step 6: If match is found, display "Match"
- Step 7: Otherwise, display "No Match"
- Step 8: Stop the program

Program

```
import re

pattern = "^[A-Za-z]+$"

string = input("Enter a string: ")

if re.match(pattern, string):
   print("Match")
else:
   print("No Match")
```

Input

Hello

Output

Match

EX NO: DATE:	DESCRIPTIVE STATISTICS BY USING NUMPY

Aim

To compute descriptive statistical measures like mean, median, standard deviation, variance, minimum, and maximum using NumPy.

Algorithm

- Step 1: Start the program
- Step 2: Import numpy module
- Step 3: Create an array of numbers
- Step 4: Calculate mean using np.mean()
- Step 5: Calculate median using np.median()
- Step 6: Calculate standard deviation using np.std()
- Step 7: Calculate variance using np.var()
- Step 8: Find minimum value using np.min()
- Step 9: Find maximum value using np.max()
- Step 10: Display all results
- Step 11: Stop the program

Program

```
import numpy as np

data = np.array([10, 20, 30, 40, 50])

mean = np.mean(data)
median = np.median(data)
std_dev = np.std(data)
variance = np.var(data)
minimum = np.min(data)
maximum = np.max(data)

print("Mean:", mean)
print("Median:", median)
print("Standard Deviation:", std_dev)
print("Variance:", variance)
```

```
print("Minimum:", minimum)
print("Maximum:", maximum)
```

Input

10, 20, 30, 40, 50

Output

Mean: 30.0
Median: 30.0
Standard Deviation: 14.1421
Variance: 200.0
Minimum: 10
Maximum: 50

EX NO: DATE:	ILLUSTRATE INDEXING OPERATION IN DATAFRAME

Aim

To illustrate different indexing operations in a Pandas DataFrame.

Algorithm

- Step 1: Start the program
- Step 2: Import pandas module
- Step 3: Create a DataFrame with sample data
- Step 4: Display the DataFrame
- Step 5: Access a single column using column name
- Step 6: Access multiple columns using list of column names
- Step 7: Access rows using iloc[] (index-based)
- Step 8: Access rows using loc[] (label-based)
- Step 9: Display selected data
- Step 10: Stop the program

Program

```
import pandas as pd

data = {
   'Name': ['A', 'B', 'C', 'D'],
   'Age': [20, 21, 22, 23],
   'Marks': [85, 90, 88, 92]
}

df = pd.DataFrame(data)

print("Original DataFrame:")
print(df)

# Access single column
```

```
print("\nName Column:")
print(df['Name'])

# Access multiple columns
print("\nName and Marks:")
print(df[['Name', 'Marks']])

# Access row using iloc
print("\nFirst Row (iloc):")
print(df.iloc[0])

# Access row using loc
print("\nRow with index 1 (loc):")
print(df.loc[1])
```

Input

Sample data:

```
Name, Age, Marks
A, 20, 85
B, 21, 90
C, 22, 88
D, 23, 92
```

Output

```
Original DataFrame:
  Name  Age  Marks
0    A   20     85
1    B   21     90
2    C   22     88
3    D   23     92

Name Column:
0    A
1    B
2    C
3    D

Name and Marks:
  Name  Marks
0    A     85
1    B     90
2    C     88
3    D     92

First Row (iloc):
Name      A
```

Age 20
Marks 85

Row with index 1 (loc):
Name B
Age 21
Marks 90

EX NO: DATE:	CREATION OF VARIOUS PLOTS USING PANDAS

Aim

To illustrate different types of plots using Pandas library.

Algorithm

- Step 1: Start the program
- Step 2: Import pandas and matplotlib.pyplot modules
- Step 3: Create a DataFrame with sample data
- Step 4: Plot a line graph using plot()
- Step 5: Plot a bar chart using plot(kind='bar')
- Step 6: Plot a histogram using plot(kind='hist')
- Step 7: Plot a scatter plot using plot(kind='scatter')
- Step 8: Display all plots using plt.show()
- Step 9: Stop the program

Program

```
import pandas as pd
import matplotlib.pyplot as plt

data = {
   'Year': [2018, 2019, 2020, 2021, 2022],
   'Sales': [100, 150, 200, 250, 300],
   'Profit': [20, 30, 50, 60, 80]
}

df = pd.DataFrame(data)

# Line Plot
df.plot(x='Year', y='Sales', kind='line', title='Line Plot')
```

```
# Bar Plot
df.plot(x='Year', y='Sales', kind='bar', title='Bar Plot')

# Histogram
df['Sales'].plot(kind='hist', title='Histogram')

# Scatter Plot
df.plot(x='Sales', y='Profit', kind='scatter', title='Scatter Plot')

plt.show()
```

Input

```
Year, Sales, Profit
2018, 100, 20
2019, 150, 30
2020, 200, 50
2021, 250, 60
2022, 300, 80
```

Output

- Line Plot showing Sales over Years
- Bar Chart of Sales
- Histogram of Sales distribution
- Scatter Plot between Sales and Profit

EX NO: DATE:	BUILDING GUI APPLICATIONS BY USING Tkinter

Aim

To create a simple GUI application using Tkinter.

Algorithm

- Step 1: Start the program
- Step 2: Import tkinter module
- Step 3: Create the main window using Tk()
- Step 4: Set window title and size
- Step 5: Add widgets like Label, Entry, and Button
- Step 6: Define function for button action
- Step 7: Place widgets on the window using pack()
- Step 8: Run the application using mainloop()
- Step 9: Stop the program

Program

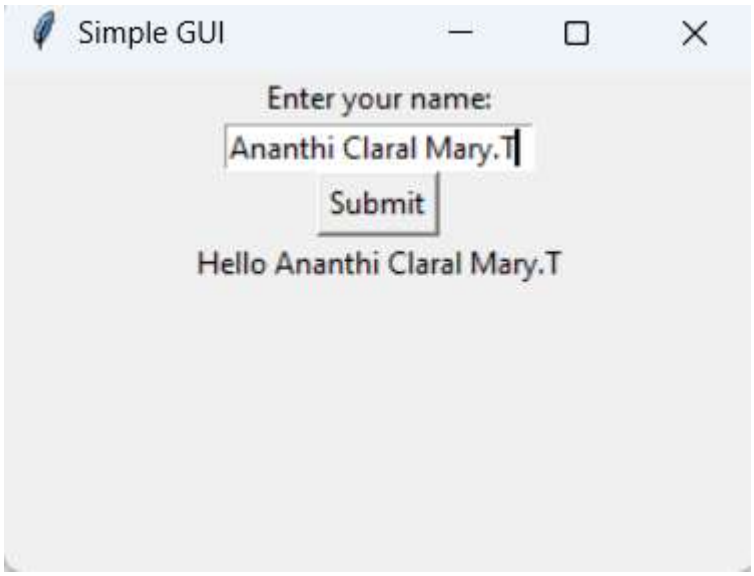
```
import tkinter as tk

def show_message():
    name = entry.get()
    label_result.config(text="Hello " + name)

# Create window
root = tk.Tk()
root.title("Simple GUI")
root.geometry("300x200")

# Widgets
```

```
label = tk.Label(root, text="Enter your name:")
label.pack()

entry = tk.Entry(root)
entry.pack()

button = tk.Button(root, text="Submit", command=show_message)
button.pack()

label_result = tk.Label(root, text="")
label_result.pack()

# Run application
root.mainloop()
```

Input

User enters name in the text box.

Output



Displays message:

Hello <username>

EX NO: DATE:	CREATING TABLES BY USING SQLITE

Aim

To create a table in SQLite database using Python.

Algorithm

- Step 1: Start the program
- Step 2: Import sqlite3 module
- Step 3: Connect to SQLite database using connect()
- Step 4: Create a cursor object using cursor()
- Step 5: Write SQL query to create a table
- Step 6: Execute the query using execute()
- Step 7: Commit the changes using commit()
- Step 8: Close the connection
- Step 9: Stop the program

Program

```
import sqlite3

# Connect to database (creates file if not exists)
conn = sqlite3.connect("student.db")

# Create cursor
cursor = conn.cursor()

# Create table
cursor.execute("""
CREATE TABLE IF NOT EXISTS Student (
    id INTEGER PRIMARY KEY,
    name TEXT,
    age INTEGER,
    marks INTEGER
)
""")

print("Table created successfully")
```

```
# Save changes
conn.commit()

# Close connection
conn.close()
```

Input

No user input required.

Output

```
Table created successfully
```

EX NO: DATE:	ILLUSTRATION ON DATABASE CONNECTIVITY

Aim

To illustrate database connectivity using Python and perform basic operations.

Algorithm

- Step 1: Start the program
- Step 2: Import sqlite3 module
- Step 3: Establish connection to the database using connect()
- Step 4: Create a cursor object using cursor()
- Step 5: Execute SQL queries (create table / insert / select)
- Step 6: Fetch data using fetchall()
- Step 7: Display the data
- Step 8: Commit the changes using commit()
- Step 9: Close the connection
- Step 10: Stop the program

Program

```
import sqlite3

# Connect to database
conn = sqlite3.connect("student.db")

# Create cursor
cursor = conn.cursor()

# Create table
cursor.execute("CREATE TABLE IF NOT EXISTS Student (id INTEGER, name TEXT)")

# Insert data
cursor.execute("INSERT INTO Student VALUES (1, 'Kader')")
cursor.execute("INSERT INTO Student VALUES (2, 'John')")

# Retrieve data
cursor.execute("SELECT * FROM Student")
```

```
rows = cursor.fetchall()

# Display data
for row in rows:
    print(row)

# Save changes
conn.commit()

# Close connection
conn.close()
```

Input

No user input required.

Output

```
(1, 'Kader')
(2, 'John')
```